ⵜⴰⵖⵉⵡⴰⵏⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ
FACULTÉ DES SCIENCES

MARRAKECH
جامعة القاضي عياض
UNIVERSITÉ CADI AYYAD

كليـــــــــة العلوم
FACULTY OF SCIENCE

Semlalia

Département Informatique
Filière :S6 Sciences de l'Informatique

Projet de Fin de Module : Programmation Backend PHP

Système de Gestion de Cabinet Médical

Réalisé par :

Amine Rihan
El Houcine Rahmouni
Zakaria Oulamine

Encadré par :

Mme Al Alaoui Hasna
Mme Baba Naima
Mme Jabir Somaya

Année Universitaire : 2025-2026

Remerciements

Avant d'entamer la présentation de ce travail, nous tenons à exprimer notre profonde gratitude et nos plus sincères remerciements à notre professeure, Mme El Alaoui Hasna, pour ses conseils précieux, sa disponibilité et son encadrement rigoureux tout au long de la réalisation de ce projet.

Nos remerciements s'étendent également à Mme Baba Naima, notre encadrante de travaux pratiques, pour son accompagnement, son soutien et ses orientations pertinentes lors des différentes étapes de ce travail.

Enfin, nous souhaitons souligner l'excellente collaboration et l'esprit d'équipe qui ont animé notre groupe de travail. Ce projet est le fruit d'un effort commun et d'un engagement partagé.

Table des Matières

Table des Matières 3
Introduction Générale 5
Chapitre 1 : 7
Contexte général du projet 7
1. Introduction 8
2. Cadre général du projet 8
2.1 Présentation du sujet 8
2.2 Équipe du projet 9
2.3 Méthodologie utilisée 9
2.4 Sprints 11
3. Conclusion 12
Chapitre 2 : 13
Analyse et Conception 13
1. Introduction 14
2. Identification des Acteurs 14
3. Modélisation Fonctionnelle 15
3.1 Diagramme des Cas d'Utilisation Général 15
3.2 Description Textuelle des Cas d'Utilisation 17
4. Modélisation Statique 20
4.1 Diagramme de Classes 20
4.1.1 Définition : 20
4.1.2 Le concept de la classe : 20
4.1.3 Dictionnaire de class et des attributs 22
5. Modélisation Dynamique 24
5.1 Définition : 24
5.2 Diagramme de séquence : Authentification d'un utilisateur 24
5.3 Diagramme de séquence : Prise de rendez-vous par un patient 25
5.4 Diagramme de séquence : Déroulement d'une consultation médicale 26
6. Conclusion 28

Table des Figures

Figure 1 : Schéma du processus Scrum 10
Figure 2 : Scrum board sous ClickUp 10
Figure 3 : Diagramme de Gantt des sprints du projet 12
Figure 4 : Diagramme des Cas d'Utilisation 16
Figure 5 : Authentification d'un utilisateur 17
Figure 6 : Prise de rendez-vous par un patient 18
Figure 7 : Déroulement d'une consultation médicale 19
Figure 8 : Diagramme de Classes 21
Figure 9 : Dictionnaire de class 23
Figure 10 : Diagramme de séquence : Authentification d'un utilisateur 25
Figure 11 : Diagramme de séquence : Prise de rendez-vous par un patient 26
Figure 12 : Diagramme de séquence : Déroulement d'une consultation médicale 27

Introduction Générale

La gestion des cabinets médicaux représente aujourd'hui l'un des défis les plus critiques dans le domaine de la santé. Une gestion efficace et numérisée permet non seulement d'optimiser le flux de travail du personnel médical et administratif, mais aussi d'améliorer significativement la qualité des soins prodigués aux patients.

Dans ce cadre, notre projet consiste à concevoir et développer une application web dédiée à la gestion d'un cabinet médical, permettant aux différents acteurs — administrateur, médecins, secrétaires et patients — d'interagir avec le système de manière fluide et sécurisée. L'application offre notamment la prise de rendez-vous en ligne, la gestion des dossiers médicaux, la génération d'ordonnances, ainsi qu'un tableau de bord statistique pour le suivi de l'activité du cabinet.

Le développement de ce projet, intitulé « Système de Gestion de Cabinet Médical », repose sur le framework Laravel (version 10.x/11.x) pour la partie backend, avec Bootstrap 5 pour le frontend, et MySQL comme système de gestion de base de données. L'architecture suit le patron MVC avec l'utilisation d'Eloquent ORM, et intègre des outils complémentaires tels que DomPDF pour l'export des ordonnances, Laravel Mail pour les notifications, et PHPUnit pour les tests unitaires. Le versionning du code est assuré via Git (GitHub/GitLab), et la méthodologie agile Scrum a encadré l'organisation du travail en sprints.

Le contenu du présent rapport est organisé en <u>quatre chapitres</u> comme suit :

Chapitre 1 : Contexte général du projet, présente le cadre général du projet, les besoins exprimés, et les différents utilisateurs du système.

Chapitre 2 : Analyse et Conception, expose les diagrammes UML réalisés, notamment le diagramme de cas d'utilisation, le diagramme de classes, ainsi que les diagrammes de séquence des principaux scénarios.

Chapitre 3 : Architecture technique, décrit les technologies et frameworks utilisés, l'architecture MVC adoptée, et les choix techniques justifiés.

Chapitre 4 : Réalisation, présente les résultats concrets du développement, les interfaces de l'application, les tests unitaires effectués, ainsi que le déploiement en ligne de la solution.

Chapitre 1 :

Contexte général du projet

1. Introduction

Ce premier chapitre présente le contexte général du projet de système de gestion de cabinet médical. Il décrit tout d'abord le cadre de réalisation du projet et les besoins auxquels l'application doit répondre, puis il précise le sujet développé, la composition de l'équipe ainsi que la méthodologie de travail adoptée (Scrum) pour mener à bien ce projet.

2. Cadre général du projet

Ce projet s'inscrit dans le contexte de la modernisation de la gestion quotidienne des cabinets médicaux, où le volume croissant de patients et de données rend les procédures manuelles peu efficaces.
L'objectif est de développer une application web de gestion de cabinet médical permettant la prise de rendez-vous en ligne, la gestion des plannings des médecins, la centralisation des dossiers médicaux des patients et la rédaction d'ordonnances de manière sécurisée.
Cette solution vise à optimiser le travail de l'administrateur, des médecins et des secrétaires, à réduire les tâches répétitives et les erreurs, et à améliorer le suivi des patients ainsi que la qualité globale du service.

2.1 Présentation du sujet

Le projet consiste à développer une application web de gestion de cabinet médical destinée à quatre types d'utilisateurs : **administrateur**, **médecins**, **secrétaires** et **patients**.
L'application permet aux **patients** de créer un compte, de consulter les disponibilités des médecins et de prendre des rendez-vous en ligne.
Les secrétaires peuvent gérer le planning des consultations, valider ou modifier les rendez-vous, et assurer le suivi administratif du cabinet.
Les médecins disposent d'un accès aux dossiers médicaux des patients, peuvent saisir les comptes rendus de consultation et générer des ordonnances au format PDF,
tandis que **l'administrateur** gère les utilisateurs, les spécialités médicales et dispose d'un tableau de bord avec des indicateurs statistiques sur l'activité du cabinet.

2.2 Équipe du projet

Ce projet a été réalisé par une équipe de trois étudiants inscrits en S6 Sciences de l'Informatique à la Faculté des Sciences Semlalia de Marrakech. L'ensemble des membres a contribué aux différentes phases du travail, de l'analyse des besoins à la conception UML, puis au développement et aux tests de l'application.

L'organisation s'est appuyée sur la répartition de petites tâches entre les membres, sans spécialisation fixe, ce qui a permis à chacun d'intervenir aussi bien sur la partie backend que sur la gestion de la base de données et l'interface utilisateur. Cette approche collaborative a favorisé une compréhension globale du système par toute l'équipe.

L'équipe du projet est composée de : **Amine Rihan**, **El Houcine Rahmouni** et **Zakaria Oulamine**.

2.3 Méthodologie utilisée

La réalisation de ce projet s'est appuyée sur la méthodologie agile Scrum, une approche itérative et incrémentale qui permet de livrer des fonctionnalités opérationnelles à chaque sprint tout en gardant une grande flexibilité face aux changements.

Scrum repose sur plusieurs éléments fondamentaux que nous avons mis en place dès le début du projet :

- **Product Backlog :**

liste complète des fonctionnalités à développer, priorisées selon les besoins du système.

- **Sprints :**

cycles courts d'une semaine, chacun ayant un objectif clair et un ensemble de tâches définies.

- **Sprint Review :**

bilan effectué à la fin de chaque sprint pour évaluer ce qui a été accompli.

- **Tableau Kanban :**

organisé en trois colonnes (To Do / In Progress / Done) pour visualiser l'avancement en temps réel.

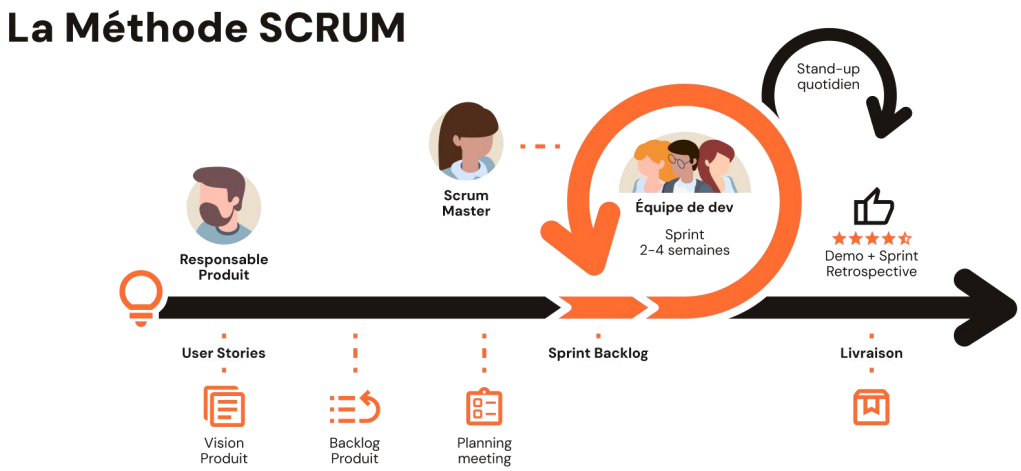

Figure 1: Schéma du processus Scrum

Pour la gestion opérationnelle des tâches, nous avons utilisé l'outil ClickUp, qui nous a permis de planifier les sprints, répartir les tâches entre les membres de l'équipe et suivre l'évolution du projet de manière collaborative et transparente.

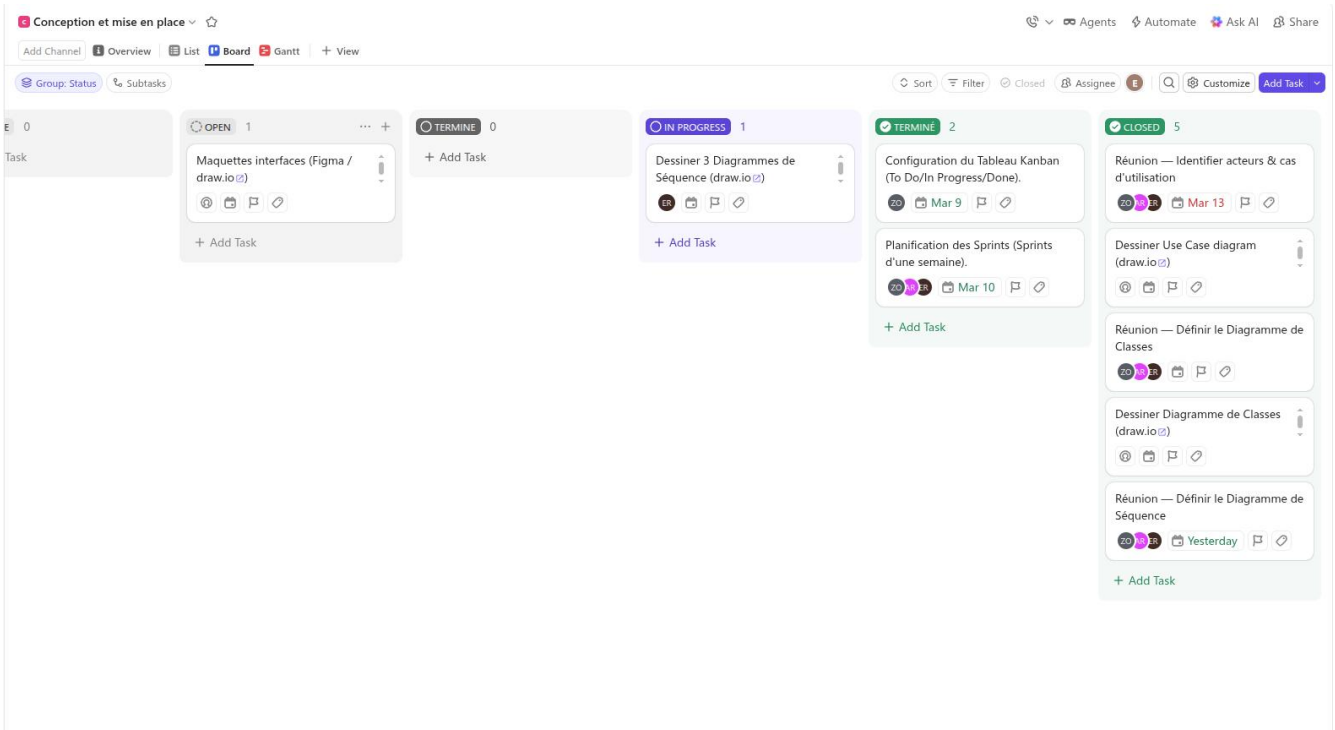

Figure 2: Scrum board sous ClickUp

2.4 Sprints

Dans le cadre de la méthodologie Scrum, le développement de l'application a été organisé en plusieurs sprints, chacun ayant des objectifs bien définis en termes de fonctionnalités à livrer. Cette organisation itérative nous a permis de prioriser les besoins, de livrer progressivement un produit fonctionnel et d'ajuster le travail en fonction des difficultés rencontrées.

Les principaux sprints réalisés peuvent être résumés comme suit :

✧ **Sprint 0 : Conception & Organisation (Space 1)**

Étude du cahier des charges et définition du Product Backlog. Réalisation des diagrammes UML obligatoires : diagramme des cas d'utilisation (acteurs : Admin, Médecin, Secrétaire, Patient), diagramme de classes (attributs, méthodes, relations) et diagrammes de séquence (authentification, rendez-vous, consultation). Configuration du tableau Kanban (To Do / In Progress / Done) et planification des sprints d'une semaine sous ClickUp..

✧ **Sprint 1 : Cœur du Système (Space 2)**

Implémentation du module d'authentification : inscription, connexion, déconnexion et gestion des rôles (Admin, Médecin, Secrétaire, Patient), avec protection CSRF et hashage des mots de passe. Développement du module de gestion des patients : CRUD complet (création, modification, recherche) et consultation de l'historique médical.

✧ **Sprint 2 : Opérations Médicales (Space 3)**

Mise en place du module de rendez-vous : affichage du calendrier des disponibilités, prise, modification et annulation de rendez-vous. Intégration des notifications par email (confirmation et rappels) via Mailtrap. Développement du module de consultations : saisie des comptes-rendus et génération des ordonnances.

✧ **Sprint 3 : Administration, Technique & Livraison (Spaces 4 & 5)**

Développement du tableau de bord administratif avec graphiques statistiques via Chart.js. Implémentation de l'export PDF des ordonnances avec DomPDF. Gestion des profils médecins et secrétaires. Création des migrations et seeders, implémentation d'Eloquent ORM, rédaction des tests

unitaires avec PHPUnit, déploiement en ligne sur Railway/Render, et finalisation du rapport de projet et du support de présentation.

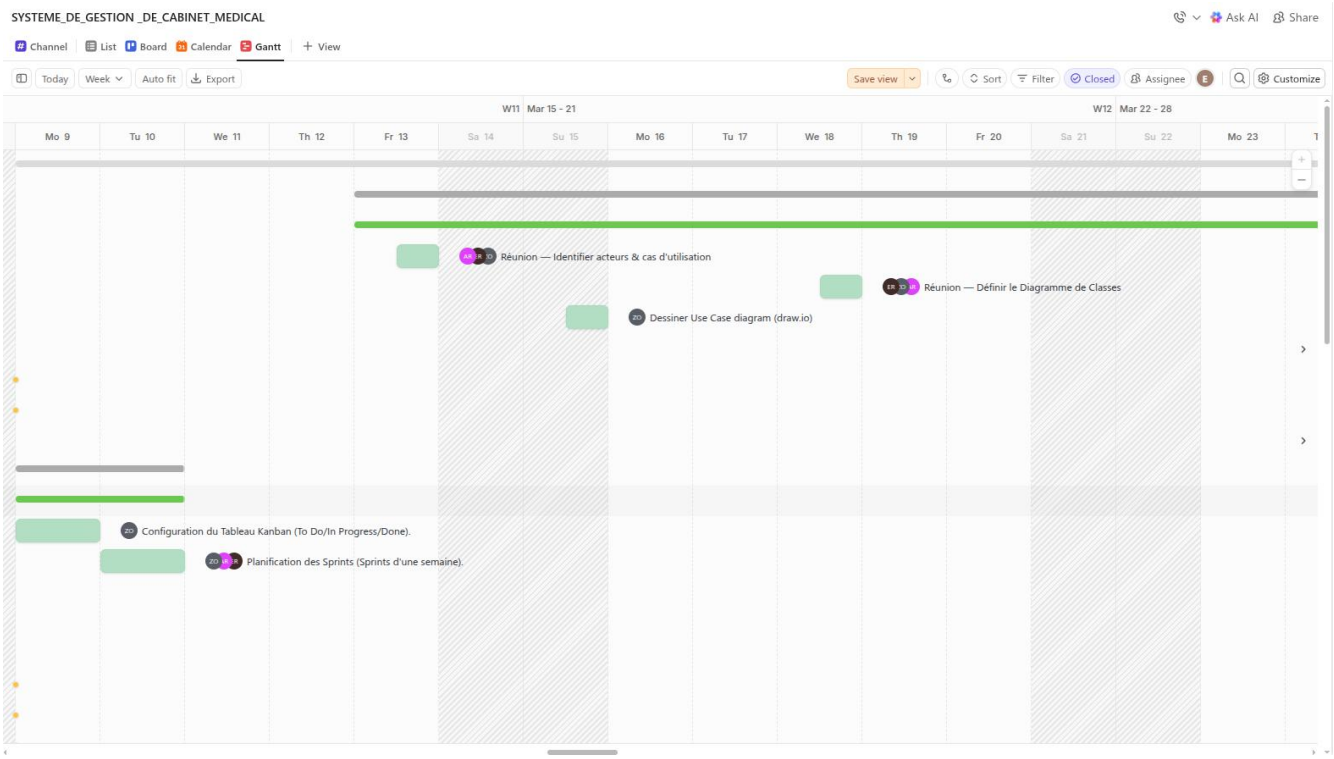

Figure 3 : Diagramme de Gantt des sprints du projet

3. Conclusion

Ce premier chapitre a présenté le contexte général du projet de système de gestion de cabinet médical, en décrivant le besoin de modernisation de la gestion quotidienne, le sujet traité ainsi que l'organisation de l'équipe. Il a également exposé la méthodologie de travail adoptée, basée sur Scrum, et la planification du développement en plusieurs sprints structurés. Les chapitres suivants détaillent successivement l'étude fonctionnelle, l'analyse et la conception, puis l'architecture technique et la réalisation de l'application.

Chapitre 2 :

Analyse et Conception

1. Introduction

Ce chapitre est consacré à l'analyse fonctionnelle et à la conception technique de notre système de gestion de cabinet médical. L'objectif principal de cette phase est de traduire les besoins fonctionnels et non fonctionnels, exprimés dans le cahier des charges, en modèles conceptuels clairs et rigoureux avant d'entamer le développement.

Pour mener à bien cette démarche, nous nous sommes appuyés sur le langage de modélisation standardisé **UML (Unified Modeling Language)**. À travers l'élaboration de différents diagrammes, UML nous permettra de cartographier les fonctionnalités du système, d'illustrer les interactions dynamiques entre les différents utilisateurs, et de concevoir une architecture logicielle et une structure de base de données optimales et évolutives.

2. Identification des Acteurs

En modélisation UML, un acteur représente toute entité externe (humain ou autre système) qui interagit directement avec l'application. Dans le cadre de notre projet de gestion de cabinet médical, nous avons identifié les acteurs suivants, structurés autour d'un concept de généralisation pour optimiser les droits d'accès :

Utilisateur (Acteur générique) :

- C'est un acteur abstrait représentant toute personne possédant un compte valide sur la plateforme.
- Il regroupe les fonctionnalités de base communes à tous les autres acteurs.
- Ses actions principales sont : l'inscription, l'authentification sécurisée et la déconnexion du système.

Administrateur :

- Il est le superviseur technique et fonctionnel de la plateforme.
- Il assure la gestion globale des comptes utilisateurs.
- Il est responsable de l'attribution stricte des rôles et des permissions.
- Il consulte le tableau de bord statistique pour le suivi de l'activité du cabinet.

Médecin :

- C'est le praticien de santé au cœur du processus médical.
- Il consulte son calendrier de rendez-vous et accède aux dossiers médicaux des patients.
- Il saisit les comptes-rendus de consultation.
- Il génère des ordonnances avec la possibilité de les exporter au format PDF.

Secrétaire :

- Elle représente le pilier administratif et l'accueil du cabinet.
- Elle gère les fiches patients (création, modification et recherche).
- Elle est chargée de la gestion centralisée des rendez-vous.
- Elle s'occupe de la configuration des plannings et des disponibilités des médecins.

Patient :

- C'est le client final du cabinet.
- Il possède un espace personnel dédié.
- Il consulte les disponibilités des médecins et gère ses rendez-vous en ligne (prise, modification ou annulation).
- Il a un accès en lecture à son propre historique médical.

3. Modélisation Fonctionnelle

Cette section est dédiée à la modélisation fonctionnelle de notre application. Elle permet de définir clairement les frontières du système et de détailler les interactions possibles entre les acteurs identifiés et la plateforme de gestion du cabinet médical.

3.1 Diagramme des Cas d'Utilisation Général

Le diagramme des cas d'utilisation donne une vue globale et structurée des fonctionnalités offertes par le système. Il met en évidence les relations (inclusion, extension, et généralisation) entre les différents cas d'utilisation et les acteurs.

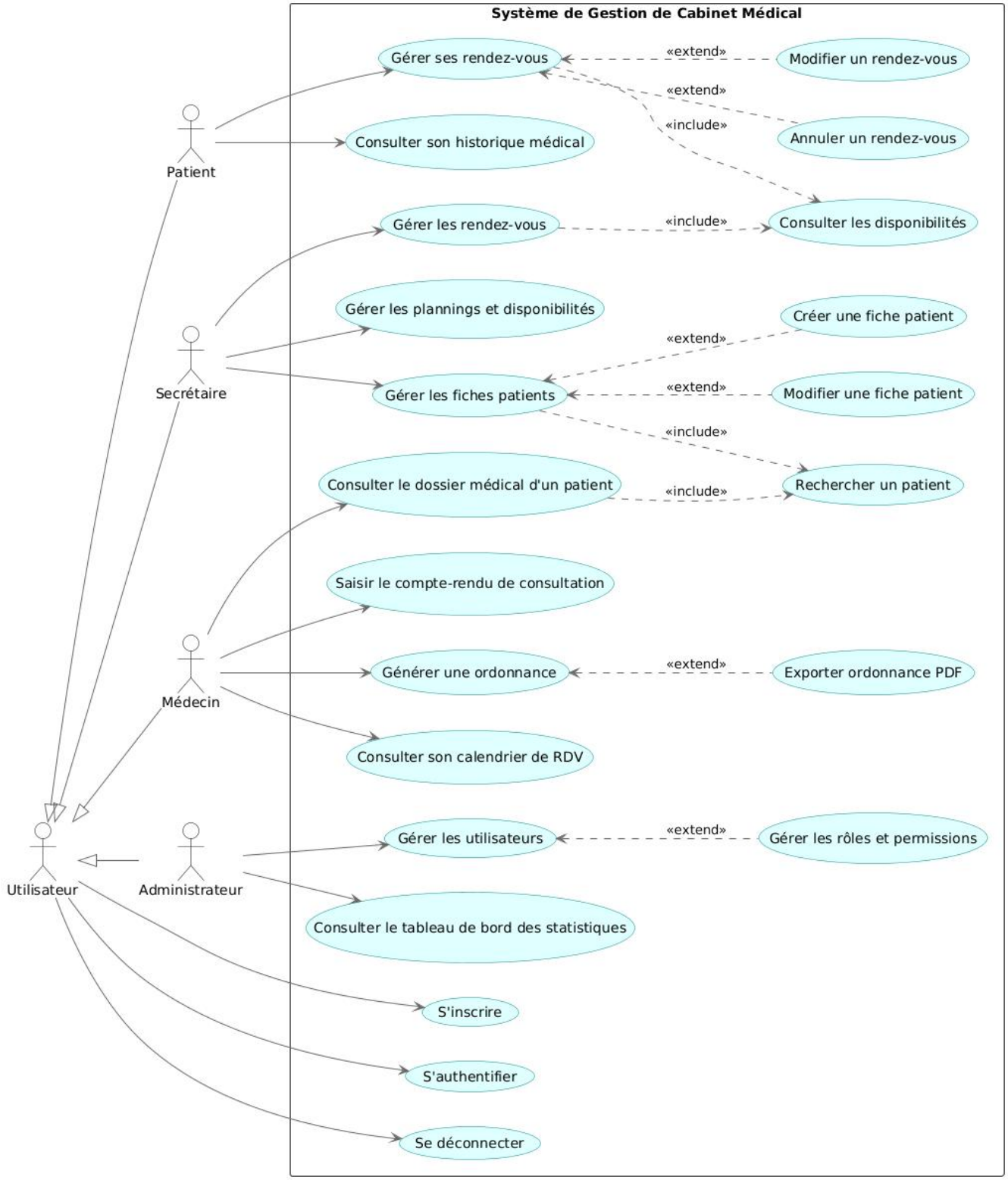

Figure 4 : Diagramme des Cas d'Utilisation

3.2 Description Textuelle des Cas d'Utilisation

Afin d'apporter plus de précision et de préparer la modélisation dynamique (diagrammes de séquence), nous avons détaillé textuellement les scénarios des cas d'utilisation les plus critiques du système.

Rubrique	Description
Cas d'utilisation	S'authentifier
Acteur principal	Utilisateur (Patient, Médecin, Secrétaire, Administrateur)
Préconditions	L'utilisateur doit posséder un compte valide (inscrit dans la base de données).
Scénario nominal	1. L'utilisateur accède à la page de connexion. 2. L'utilisateur saisit ses identifiants (Email et Mot de passe) et valide. 3. Le système vérifie les informations saisies dans la base de données. 4. Le système identifie le rôle de l'utilisateur. 5. Le système redirige l'utilisateur vers son tableau de bord spécifique (Espace Patient, Médecin, etc.).
Scénario alternatif	Si les identifiants sont incorrects, le système affiche un message d'erreur et demande une nouvelle saisie.
Post-conditions	L'utilisateur est connecté et sa session est active.

Figure 5 : Authentification d'un utilisateur

Rubrique	Description
Cas d'utilisation	Prendre un rendez-vous en ligne
Acteur principal	Patient
Préconditions	Le patient doit être authentifié sur la plateforme.
Scénario nominal	1. Le patient clique sur "Prendre un rendez-vous". 2. Le système affiche la liste des médecins et leurs spécialités. 3. Le patient sélectionne un médecin. 4. Le système affiche le calendrier avec les disponibilités du médecin (cas inclus : Consulter disponibilités). 5. Le patient choisit un créneau horaire disponible et confirme. 6. Le système enregistre le rendez-vous avec le statut "Confirmé" ou "En attente". 7. Le système déclenche l'envoi d'un email de confirmation au patient.
Scénario alternatif	Si le créneau choisi vient juste d'être réservé par un autre patient ou la secrétaire, le système affiche un message d'erreur et rafraîchit le calendrier.
Post-conditions	Le planning du médecin est mis à jour et le rendez-vous est ajouté à l'historique du patient.

Figure 6 : Prise de rendez-vous par un patient

Rubrique	Description
Cas d'utilisation	Saisir le compte-rendu et générer une ordonnance
Acteur principal	Médecin
Préconditions	Le médecin doit être authentifié et avoir sélectionné un patient ayant un rendez-vous ce jour-là.
Scénario nominal	1. Le médecin accède au dossier médical du patient (cas inclus : Rechercher un patient). 2. Le médecin consulte l'historique médical pour préparer la consultation. 3. Après l'examen, le médecin saisit le compte-rendu de la consultation dans le formulaire dédié. 4. Le médecin saisit les médicaments et génère une ordonnance. 5. Le système sauvegarde le compte-rendu et l'ordonnance dans le dossier du patient.
Scénario alternatif	Lors de l'étape 4, le médecin peut choisir l'option d'exporter l'ordonnance en format PDF (cas étendu : Exporter ordonnance PDF) pour l'imprimer ou l'envoyer au patient.
Post-conditions	Le dossier médical du patient est mis à jour avec la nouvelle consultation et l'ordonnance.

Figure 7 : Déroulement d'une consultation médicale

4. Modélisation Statique

4.1 Diagramme de Classes

4.1.1 Définition :

Les diagrammes de class sans doute c'est les diagrammes les plus utiliser d'UML ils décrivent les type des objets qui composent un système et les différent type de relation statique qui existent entre eux.

4.1.2 Le concept de la classe :

Une classe est une description d'un groupe d'objets partageant un ensemble commun de propriétés (les attributs), de comportements (les opérations) et de relations avec d'autres objets (les associations et les agrégations).

La classe est définie par son nom, ses attributs et ses opérations.

Les classes sur lesquelles se porte mon application sont les suivantes :

- User
- Role
- Medecin
- Secretaire
- Admin
- DossierMedical
- Disponibilite
- RendezVous
- Consultation
- Ordonnance
- Notification

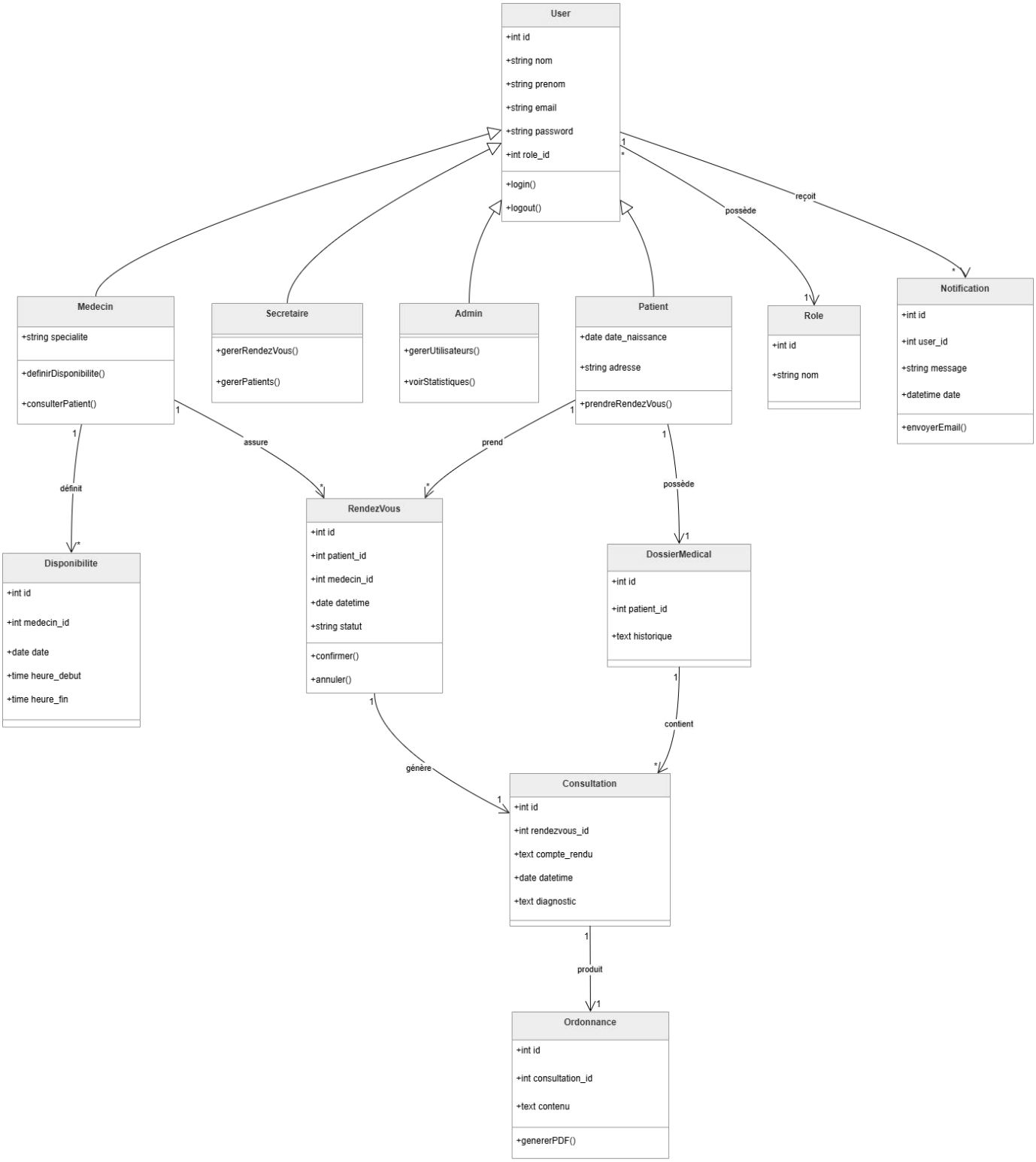

Figure 8 : Diagramme de Classes

4.1.3 Dictionnaire de class et des attributs

Les classes	L'attribut (Le champ)	L'attribut (Le type)	Méthode
User	**<u>id</u>** nom prenom email password role_id	Numérique Caractère Caractère Caractère Caractère Numérique	login() logout()
Role	**<u>id</u>** nom	Numérique Caractère	
Patient	date_naissance adresse	Date Caractère	prendreRendezVous()
Medecin	specialite	Caractère	definirDisponibilite() consulterPatient()
Secrétaire	*(Hérite de User)*	-	gererRendezVous() gererPatients()
Admin	*(Hérite de User)*	-	gererUtilisateurs() voirStatistiques()
RendezVous	**<u>id</u>** patient_id medecin_id date heure statut	Numérique Numérique Numérique Date Heure Caractère	confirmer() annuler()

Les classes	L'attribut (Le champ)	L'attribut (Le type)	Méthode
Disponibilite	**<u>id</u>** medecin_id date heure_debut heure_fin	Numérique Numérique Date Heure Heure	
DossierMedical	**<u>id</u>** patient_id historique	Numérique Numérique Texte	
Consultation	**<u>id</u>** rendezvous_id compte_rendu datetime diagnostic	Numérique Numérique Texte Date/Heure Texte	
Ordonnance	**<u>id</u>** consultation_id contenu	Numérique Numérique Texte	genererPDF()
Notification	**<u>id</u>** user_id message date	Numérique Numérique Caractère Date/Heure	envoyerEmail()

Figure 9 : Dictionnaire de class

5. Modélisation Dynamique

Cette section est consacrée à la modélisation dynamique de notre système de gestion de cabinet médical.

Contrairement au diagramme de classes qui décrit la structure statique, les diagrammes de séquence permettent de représenter le comportement du système en illustrant les échanges de messages entre les différents acteurs et composants, ordonnés chronologiquement selon un scénario précis.

5.1 Définition :

Le diagramme de séquence est l'un des diagrammes comportementaux de la notation **UML**. Il modélise les interactions entre les acteurs et les objets du système en les ordonnant de haut en bas selon l'axe du temps.

Chaque participant est représenté par une ligne de vie (lifeline), et les échanges entre eux sont matérialisés par des flèches de messages. Les blocs conditionnels (alt, opt) permettent de représenter les scénarios alternatifs et les comportements optionnels du système.

5.2 Diagramme de séquence : Authentification d'un utilisateur

Ce diagramme illustre le processus d'authentification dans notre application. Il représente les interactions entre l'utilisateur, l'interface de connexion, le contrôleur **AuthController** et la base de données, depuis la saisie des identifiants jusqu'à la redirection vers le tableau de bord correspondant au rôle identifié.

Le scénario alternatif — identifiants incorrects — est représenté par un bloc conditionnel alt.

Figure 10 : Diagramme de séquence : Authentification d'un utilisateur

5.3 Diagramme de séquence : Prise de rendez-vous par un patient

Ce diagramme détaille le processus de prise de rendez-vous en ligne par un patient authentifié. Il met en évidence les interactions entre le patient, le **RendezVousController**, le **DisponibiliteController**, la base de données et le service de messagerie Laravel Mail.

La relation d'inclusion avec le cas « **Consulter disponibilités** » est représentée, ainsi que le scénario alternatif dans lequel le créneau sélectionné a déjà été réservé par un autre utilisateur.

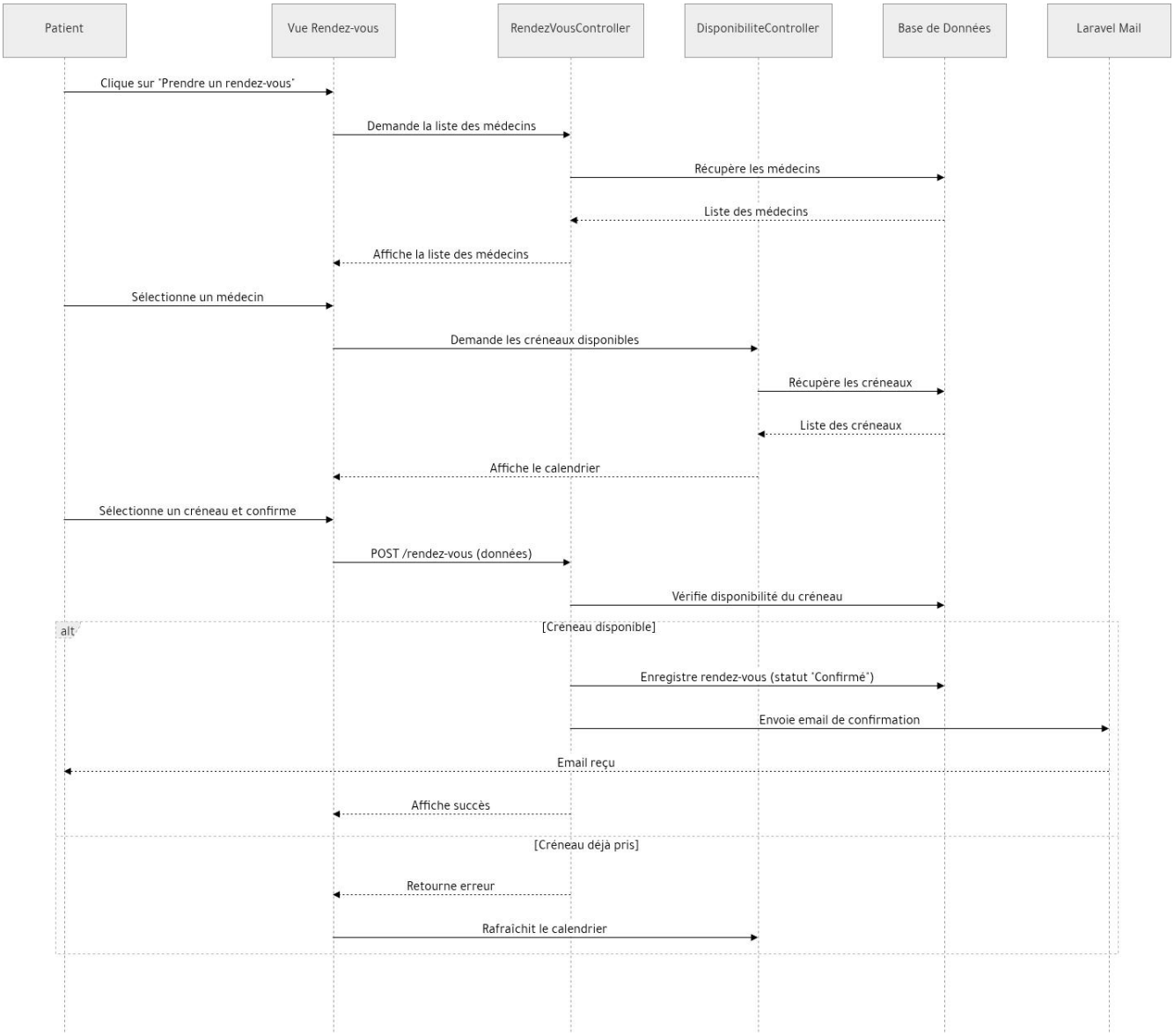

Figure 11 : Diagramme de séquence : Prise de rendez-vous par un patient

5.4 Diagramme de séquence : Déroulement d'une consultation médicale

Ce diagramme modélise le déroulement complet d'une consultation médicale, de l'accès au dossier du patient jusqu'à la génération de l'ordonnance. Il représente les interactions entre le médecin, le **ConsultationController**, la base de données et le service d'export PDF **DomPDF**.

Le bloc **opt** en fin de séquence illustre la relation d'extension avec le cas « Exporter ordonnance PDF ».

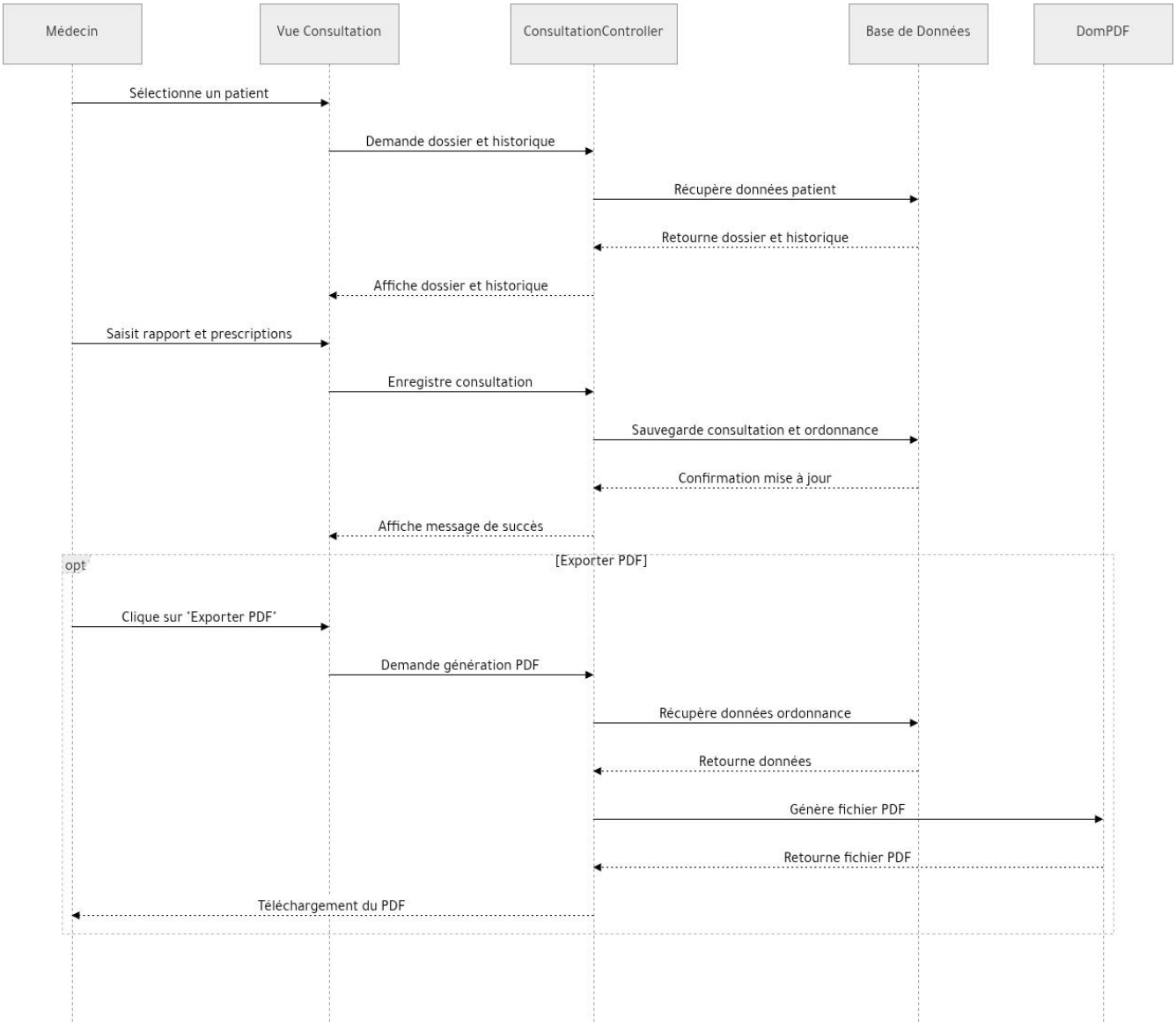

Figure 12 : Diagramme de séquence : Déroulement d'une consultation médicale

6. Conclusion

Ce deuxième chapitre a été consacré à l'analyse et à la conception de notre système de gestion de cabinet médical. Nous avons commencé par identifier les différents acteurs du système — Administrateur, Médecin, Secrétaire et Patient — en précisant leurs rôles et leurs interactions avec la plateforme.

Nous avons ensuite procédé à la modélisation fonctionnelle à travers le diagramme des cas d'utilisation, qui offre une vue globale des fonctionnalités du système et met en évidence les relations d'inclusion, d'extension et de généralisation entre les acteurs et les cas d'utilisation. Cette modélisation a été complétée par une description textuelle détaillée des scénarios les plus critiques.

La modélisation dynamique, réalisée à travers trois diagrammes de séquence, a permis d'illustrer le comportement du système lors des interactions clés : l'authentification d'un utilisateur, la prise de rendez-vous par un patient et le déroulement d'une consultation médicale.

Enfin, la modélisation statique, matérialisée par le diagramme de classes et le dictionnaire de données, a défini la structure interne du système, ses entités et les relations qui les lient.

L'ensemble de ces modèles constitue une base solide pour la phase de développement. Le chapitre suivant présentera l'architecture technique adoptée ainsi que les choix technologiques retenus pour la réalisation de l'application.